Báo cáo bài tập E1–E3

Học sâu và ứng dụng trong thị giác máy tính
Mã môn học: CO5085, Học kỳ 261 (2026–2027)

Nguyễn Xuân Trường (2570526)
Trần Lê Hoàng Long (2570443)

Cập nhật lần cuối: 01/11/2026

Giảng viên hướng dẫn: Lê Thành Sách, Nguyễn Quốc Minh
Repository: https://github.com/truong1501/SDH_HK261_CO5085

Bản này gồm E1, E2 và E3.

Mục lục

1 Dữ liệu và protocol dùng chung cho E1–E3 **3**
1.1 Tập dữ liệu: Fashion-MNIST . . . 3
1.2 Split train / validation / test . . . 3
1.3 Tiền xử lý và augmentation . . . 5
1.4 Vòng lặp huấn luyện dùng chung . . . 6
1.5 Cách đánh giá và môi trường . . . 6

2 E1: Phân loại ảnh với softmax, MLP và CNN **7**
2.1 Mục tiêu . . . 7
2.2 Kiến trúc ba mô hình . . . 7
2.3 Cấu hình huấn luyện . . . 8
2.4 Kết quả . . . 8
2.5 Phân tích lỗi . . . 10
2.6 Nhận xét . . . 13
2.7 Hạn chế . . . 14

3 E2: Transformer, MSA và tokenization **15**
3.1 Mục tiêu và phạm vi . . . 15
3.2 Kiến trúc và công thức . . . 15
3.3 Bốn cách lấy token . . . 16
3.4 Thiết kế thí nghiệm và huấn luyện . . . 16
3.5 Kiểm chứng MSA tự viết . . . 17
3.6 Kết quả so sánh . . . 17
3.7 Phân tích lỗi và hình attention . . . 19
3.8 Kết luận E2 . . . 20

4 So sánh E1 và E2 **21**

5 E3: LSTM và GRU trên chuỗi ảnh **22**
5.1 Mục tiêu . . . 22
5.2 Hai cách biến ảnh thành chuỗi . . . 22
5.3 Kiến trúc . . . 22
5.4 Cấu hình huấn luyện . . . 23
5.5 Kết quả . . . 23
5.6 Hạn chế . . . 24

Phụ lục **25**

Tài liệu tham khảo **26**

1 Dữ liệu và protocol dùng chung cho E1–E3

1.1 Tập dữ liệu: Fashion-MNIST

Cả ba bài E1, E2, E3 dùng chung một tập ảnh nhỏ là Fashion-MNIST [1]: ảnh xám 28×28, 10 lớp trang phục, mỗi lớp cân bằng số lượng ảnh, gồm 60.000 ảnh train và 10.000 ảnh test.

Bảng 1: Mười lớp của Fashion-MNIST

ID	Nhãn	Diễn giải
0	T-shirt/top	Áo thun
1	Trouser	Quần dài
2	Pullover	Áo chui đầu
3	Dress	Váy liền
4	Coat	Áo khoác
5	Sandal	Dép xăng đan
6	Shirt	Áo sơ mi
7	Sneaker	Giày thể thao
8	Bag	Túi
9	Ankle boot	Bốt cổ thấp

Nhóm chọn Fashion-MNIST thay vì MNIST vì MNIST quá dễ, cả ba kiến trúc softmax/MLP/CNN thường đạt accuracy rất cao và gần nhau (trên 97%), khó thấy rõ chênh lệch về khả năng biểu diễn giữa các kiến trúc cũng như khó phân tích trường hợp sai. Fashion-MNIST giữ nguyên kích thước và số lớp như MNIST nên không làm tăng chi phí tính toán, nhưng khó hơn đáng kể (nhiều lớp trang phục có hình dạng tương tự nhau khi nhìn ở độ phân giải thấp), nên chênh lệch giữa ba mô hình thể hiện rõ và có nhiều trường hợp sai đáng để phân tích. Nhóm không chọn CIFAR-10 vì ảnh màu 3 kênh sẽ làm thời gian huấn luyện tăng đáng kể, trong khi quỹ thời gian cho E1–E3 eo hẹp hơn nhiều so với bài tập lớn A1.

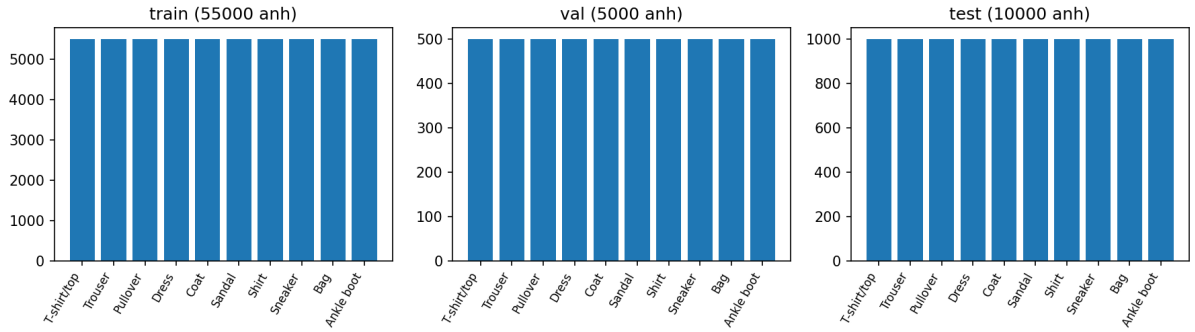
1.2 Split train / validation / test

Fashion-MNIST có sẵn 60.000 ảnh train và 10.000 ảnh test. Nhóm giữ nguyên tập test chính thức để đánh giá cuối cùng, và tách 5.000 ảnh (500 ảnh/lớp) từ tập train làm validation, chia theo kiểu stratified (đều nhau giữa các lớp), seed cố định 42.

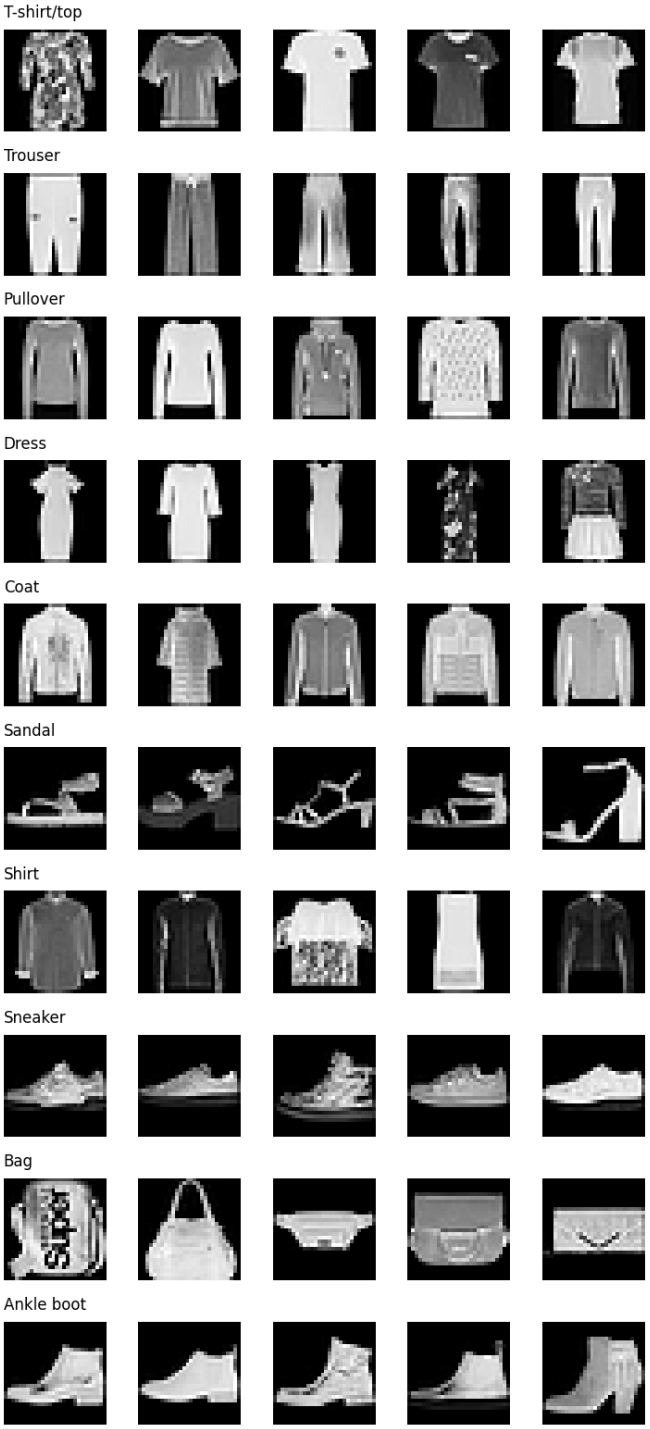
Bảng 2: Split dữ liệu dùng cho E1–E3

Split	Ảnh/lớp	Tổng	Dùng để
Train	5.500	55.000	Huấn luyện
Validation	500	5.000	Chọn checkpoint tốt nhất
Test	1.000	10.000	Báo cáo kết quả cuối, chỉ đánh giá một lần

Chỉ số ảnh của từng split được sinh bằng một hàm chia stratified cố định seed, lưu lại trong thư mục `E1/splits/` của repository để mọi thành viên trong nhóm và mọi bài E1–E3 dùng chung đúng một split, tái lập được.



Hình 1: Số ảnh mỗi lớp theo ba split. Các lớp có số ảnh bằng nhau, nên accuracy không bị lệch về một lớp đông ảnh



Hình 2: Năm ảnh train đầu tiên của mỗi lớp. Ở độ phân giải này, các lớp áo (T-shirt/top, Pullover, Coat, Shirt) có đường viền khá giống nhau; mục 2.5 và 3.7 đối chiếu nhận xét này với confusion matrix

1.3 Tiền xử lý và augmentation

Ảnh được chuẩn hóa theo mean/std tính trên tập train của Fashion-MNIST (mean $= 0{,}2860$, std $= 0{,}3530$). Khi huấn luyện, ảnh train được thêm `RandomCrop(28, padding=2)` để tăng tính đa dạng về vị trí vật thể trong ảnh. Nhóm không dùng lật ngang (horizontal flip) vì nhiều lớp trong Fashion-MNIST có hướng cố định (giày, dép, túi xách), lật ngang có thể tạo ra ảnh

không còn hợp lý về mặt vật lý. Tập validation và test không dùng augmentation, chỉ chuẩn hóa.

1.4 Vòng lặp huấn luyện dùng chung

Theo quy định chung của môn học, nhóm tự viết vòng lặp huấn luyện bằng PyTorch thay vì gọi API huấn luyện cấp cao. Mỗi epoch, với từng batch, thứ tự thực hiện là: tính logits qua mô hình (forward), tính loss bằng `CrossEntropyLoss` (hàm này đã gộp sẵn softmax và negative log-likelihood), gọi `loss.backward()` để lan truyền ngược rồi gọi `optimizer.step()` để cập nhật trọng số. Vòng lặp này dùng chung cho cả ba mô hình của E1, chỉ khác nhau ở kiến trúc được truyền vào.

1.5 Cách đánh giá và môi trường

Mỗi mô hình lấy checkpoint có validation accuracy cao nhất, rồi đánh giá một lần trên test. Macro-F1 là trung bình F1 của mười lớp. Mỗi cấu hình chỉ chạy một lần với seed 42, nên các chênh lệch nhỏ chưa đủ để kết luận chắc chắn (xem mục 2.7 và 3.6).

Bảng 3: Môi trường chạy của E1 và E2

Mục	E1	E2
Nền tảng	Google Colab	Google Colab
GPU	NVIDIA Tesla T4	NVIDIA A100-SXM4-40GB
PyTorch	2.11.0+cu130	2.11.0+cu130

Hai bài chạy trên GPU khác nhau, nên thời gian huấn luyện của E1 và E2 không so trực tiếp được với nhau; chỉ so sánh thời gian trong nội bộ từng bài.

2 E1: Phân loại ảnh với softmax, MLP và CNN

2.1 Mục tiêu

Xây dựng và so sánh ba bộ phân loại trên Fashion-MNIST: một bộ softmax tuyến tính trên vector ảnh đã flatten, một mạng MLP nhiều tầng cũng trên vector flatten, và một mạng CNN có convolution/pooling. Mục tiêu là thấy rõ ảnh hưởng của capacity và cấu trúc kiến trúc (có hay không có inductive bias không gian của convolution) lên khả năng phân loại.

2.2 Kiến trúc ba mô hình

Cả ba mô hình dùng đầu vào là ảnh $1 \times 28 \times 28$ và đầu ra là 10 lớp.

Softmax classifier. Flatten ảnh thành vector 784 chiều rồi ánh xạ tuyến tính trực tiếp sang 10 lớp: `Flatten → Linear(784, 10)`. Softmax nằm trong hàm loss, không tách riêng thành một tầng. Tổng số tham số: $784 \times 10 + 10 = 7.850$.

MLP classifier. Flatten ảnh rồi qua hai tầng ẩn có ReLU và Dropout chống overfit: `Flatten → Linear(784, 256) → ReLU → Dropout(0,2) → Linear(256, 128) → ReLU → Dropout(0,2) → Linear(128, 10)`. Tổng số tham số: $(784{\times}256{+}256){+}(256{\times}128{+}128){+}(128{\times}10{+}10) = 235.146$.

CNN classifier. Hai khối Conv–BatchNorm–ReLU–MaxPool để trích đặc trưng không gian, sau đó một tầng ẩn fully-connected làm head phân loại: `Conv2d(1, 32, 3) → BN → ReLU → MaxPool2 → Conv2d(32, 64, 3) → BN → ReLU → MaxPool2 → Flatten → Linear(3136, 128) → ReLU → Dropout(0,3) → Linear(128, 10)`. Ảnh giảm kích thước từ $28{\times}28$ xuống $7{\times}7$ sau hai lần pooling, với 64 kênh đặc trưng ở tầng cuối. Tổng số tham số: 421.834.

Bảng 4 liệt kê kích thước tensor qua từng bước của CNN, với B là số ảnh trong batch, để thấy rõ ảnh bị thu nhỏ không gian ($28{\times}28 \to 14{\times}14 \to 7{\times}7$) trong khi số kênh tăng lên ($1 \to 32 \to 64$), đúng mẫu hình chung của CNN là đánh đổi độ phân giải không gian lấy độ sâu đặc trưng.

Bảng 4: Kích thước tensor của CNN qua từng bước

Bước	Kích thước
Ảnh đầu vào	$B \times 1 \times 28 \times 28$
Sau Conv2d(1, 32, 3) + BN + ReLU	$B \times 32 \times 28 \times 28$
Sau MaxPool2	$B \times 32 \times 14 \times 14$
Sau Conv2d(32, 64, 3) + BN + ReLU	$B \times 64 \times 14 \times 14$
Sau MaxPool2	$B \times 64 \times 7 \times 7$
Sau Flatten	$B \times 3.136$
Sau Linear(3136, 128) + ReLU + Dropout	$B \times 128$
Logits (Linear(128, 10))	$B \times 10$

Bảng 5: Số tham số của ba mô hình

Mô hình	Số tham số
Softmax	7.850
MLP	235.146
CNN	421.834

MLP có capacity gấp khoảng 30 lần softmax, CNN có capacity gấp khoảng 54 lần softmax nhưng vẫn ít hơn MLP tính thuần theo số tham số FC; phần capacity của CNN chủ yếu nằm ở tầng FC cuối (3136 → 128), các tầng convolution chỉ chiếm một phần nhỏ nhờ cơ chế chia sẻ trọng số (weight sharing) của convolution.

2.3 Cấu hình huấn luyện

Cả ba mô hình dùng chung optimizer Adam, learning rate 10^{-3}, batch size 128, seed 42. Số epoch khác nhau vì tốc độ hội tụ khác nhau giữa các kiến trúc, chọn checkpoint có validation accuracy cao nhất để đánh giá trên test.

Bảng 6: Cấu hình huấn luyện từng mô hình

Mô hình	Optimizer	Learning rate	Batch size	Epoch	Seed
Softmax	Adam	10^{-3}	128	20	42
MLP	Adam	10^{-3}	128	20	42
CNN	Adam	10^{-3}	128	15	42

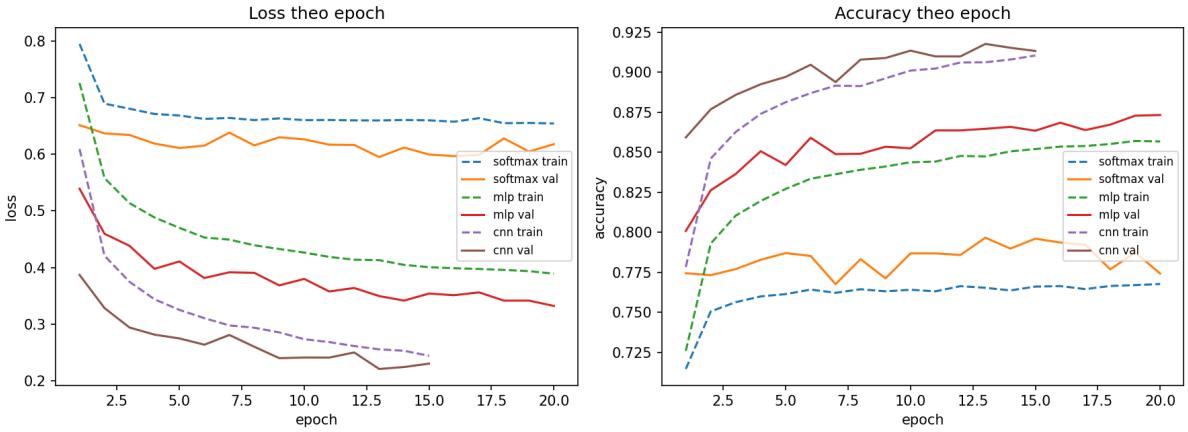
Thí nghiệm chạy trên Google Colab, GPU Tesla T4. Thời gian huấn luyện của cả ba mô hình đều dưới 5 phút nhờ Fashion-MNIST là tập ảnh nhỏ.

2.4 Kết quả

Bảng 7: So sánh ba mô hình trên tập test (10.000 ảnh)

Mô hình	Số tham số	Val acc.	Test acc.	Macro-F1	Thời gian (giây)
Softmax	7.850	0,7966	0,7905	0,7878	295,0
MLP	235.146	0,8732	0,8739	0,8736	299,7
CNN	421.834	0,9176	0,9145	0,9139	236,2

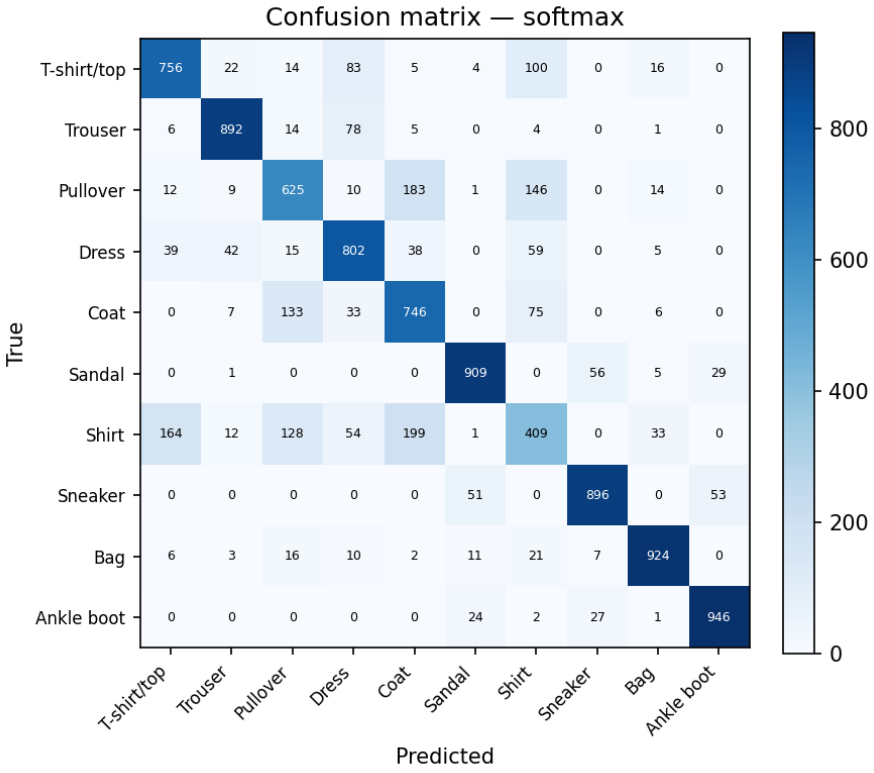
Accuracy tăng dần đều theo capacity và độ phức tạp kiến trúc: softmax đạt 79,05%, MLP đạt 87,39% (tăng 8,3 điểm phần trăm so với softmax), CNN đạt 91,45% (tăng thêm 4,1 điểm phần trăm so với MLP). Tính theo số ảnh tuyệt đối trên 10.000 ảnh test, softmax đúng 7.905 và sai 2.095 ảnh, MLP đúng 8.739 và sai 1.261 ảnh, CNN đúng 9.145 và sai 855 ảnh. CNN vừa chính xác nhất vừa huấn luyện nhanh nhất trong ba mô hình, dù chạy ít epoch hơn vẫn hội tụ tốt hơn. Điều này cho thấy inductive bias không gian của convolution giúp mô hình học hiệu quả hơn trên dữ liệu ảnh, thay vì chỉ tăng số tham số như MLP.



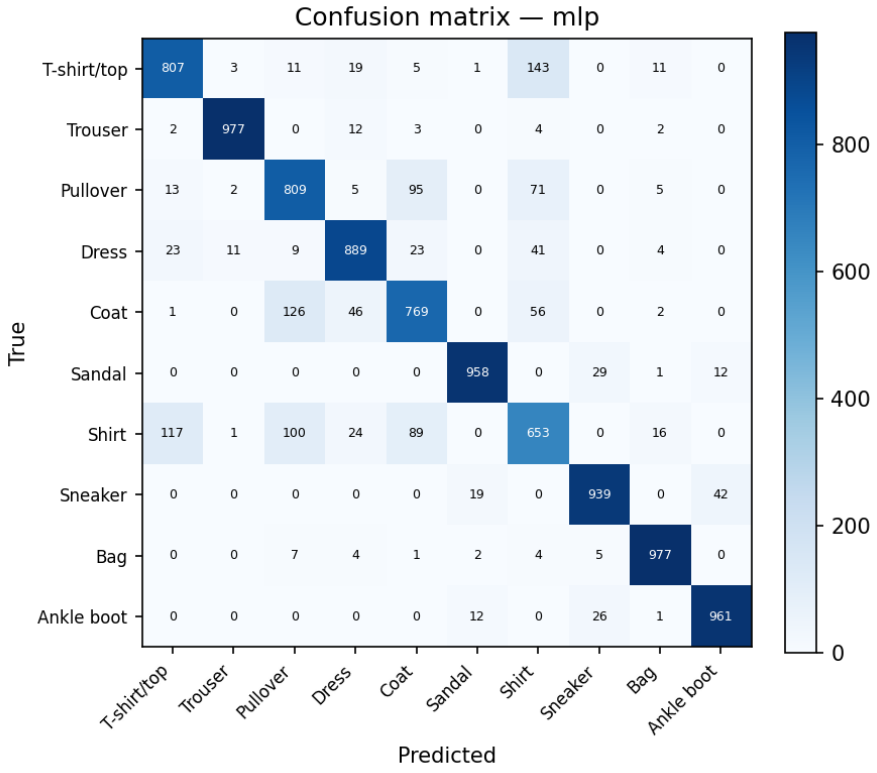
Hình 3: Đường cong loss và accuracy theo epoch của ba mô hình (nét đứt: train, nét liền: validation)

Trên Hình 3, ở cả MLP và CNN, accuracy trên tập train trong lúc huấn luyện lại thấp hơn một chút so với validation, mặc dù thông thường train accuracy phải cao hơn nếu mô hình overfit. Nguyên nhân là Dropout chỉ được bật khi mô hình ở chế độ train: trong lúc tính train accuracy, một phần neuron bị tắt ngẫu nhiên nên dự đoán khó hơn, trong khi lúc đánh giá validation, Dropout tắt hoàn toàn nên mô hình dùng toàn bộ năng lực đã học được. Đây không phải dấu hiệu underfit mà là hệ quả bình thường của Dropout, và cũng cho thấy ở mức epoch đã chạy, cả MLP và CNN chưa có dấu hiệu overfit rõ rệt (val loss vẫn đang giảm hoặc đi ngang chứ chưa tăng trở lại). Riêng softmax, do capacity rất thấp, loss và accuracy trên cả train lẫn validation bão hòa sớm và dao động nhẹ quanh một mức cố định suốt 20 epoch, đặc trưng của underfitting hơn là overfitting.

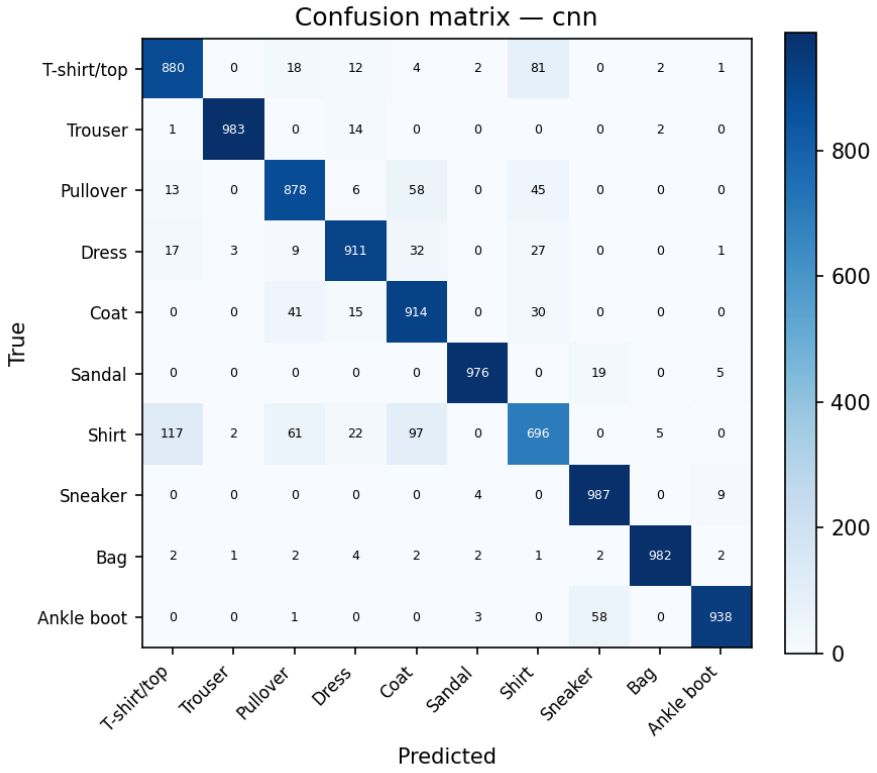
2.5 Phân tích lỗi



Hình 4: Confusion matrix của softmax trên tập test



Hình 5: Confusion matrix của MLP trên tập test



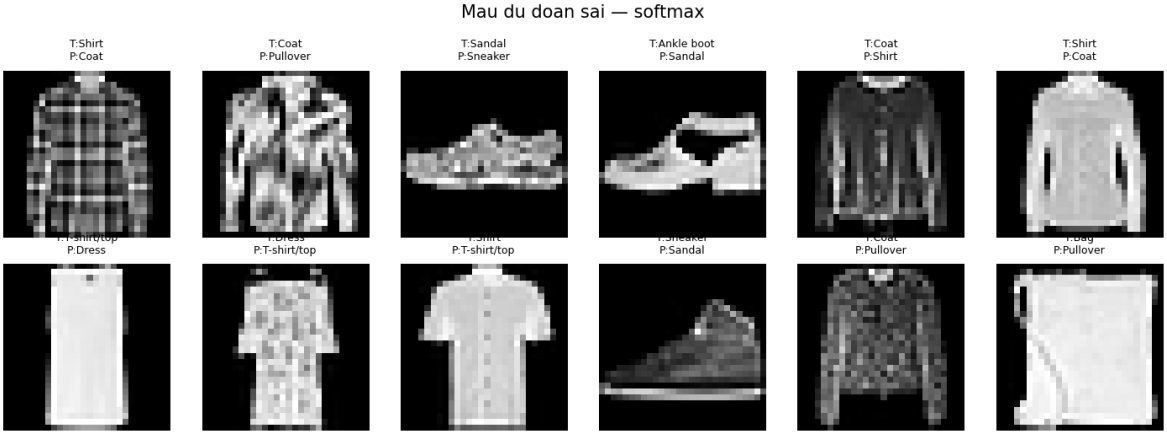
Hình 6: Confusion matrix của CNN trên tập test

Cả ba mô hình đều mắc lỗi tập trung ở cùng một cụm bốn lớp trang phục phần thân trên: T-shirt/top, Pullover, Coat và Shirt. Đây là các lớp có hình dạng tổng thể giống nhau (đều là áo nhìn thẳng, không tay hoặc có tay), khác nhau chủ yếu ở chi tiết nhỏ như độ dày vải hay cổ áo, vốn khó phân biệt ở ảnh xám 28×28. Lớp khó nhất với cả ba mô hình là **Shirt**:

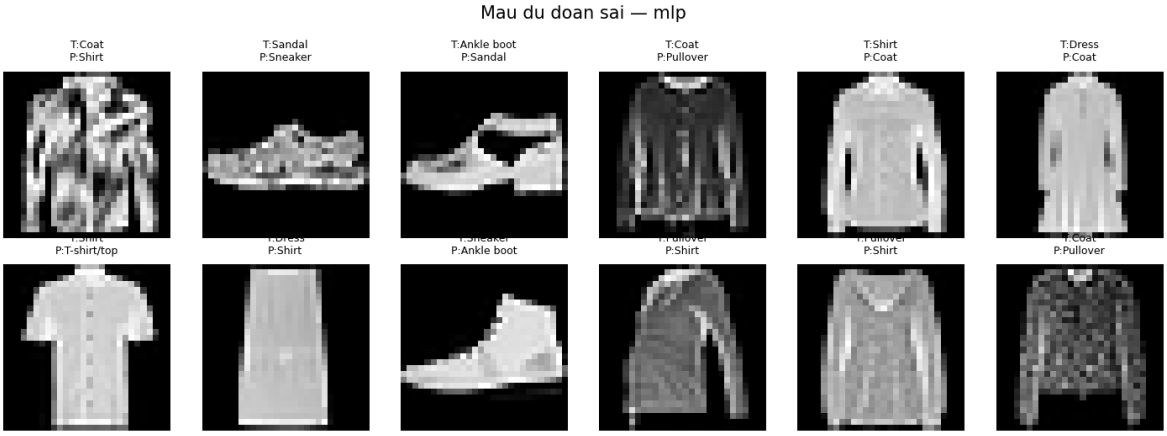
- Softmax chỉ nhận đúng 409/1.000 ảnh Shirt (40,9%), phần lớn bị nhầm thành Coat (199 ảnh), Pullover (128 ảnh) và T-shirt/top (164 ảnh).
- MLP nhận đúng 653/1.000 ảnh Shirt (65,3%), lỗi giảm nhiều nhưng vẫn nhầm sang T-shirt/top (117 ảnh), Pullover (100 ảnh) và Coat (89 ảnh).
- CNN nhận đúng 696/1.000 ảnh Shirt (69,6%), vẫn là lớp yếu nhất của CNN nhưng đã tốt hơn MLP, lỗi còn lại chủ yếu vẫn rơi vào đúng ba lớp T-shirt/top, Pullover, Coat.

Ngược với Shirt, lớp dễ nhất cũng khác nhau giữa ba mô hình: Ankle boot đạt recall cao nhất với softmax (946/1.000, 94,6%), Bag và Trouser cùng cao nhất với MLP (977/1.000, 97,7% mỗi lớp), còn Sneaker cao nhất với CNN (987/1.000, 98,7%). Khoảng cách giữa lớp dễ nhất và lớp khó nhất (Shirt) thu hẹp dần theo capacity: từ 53,7 điểm phần trăm ở softmax (94,6% so với 40,9%) xuống còn 29,1 điểm ở CNN (98,7% so với 69,6%), cho thấy phần lớn chênh lệch accuracy giữa ba mô hình đến từ việc xử lý tốt hơn đúng cụm lớp khó, chứ không phải cải thiện đều trên mọi lớp.

Chiều ngược lại cũng xảy ra: T-shirt/top hay bị cả ba mô hình dự đoán nhầm thành Shirt (100 ảnh ở softmax, 143 ảnh ở MLP, 81 ảnh ở CNN), và Pullover hay bị nhầm thành Coat. Điều này cho thấy ranh giới quyết định giữa bốn lớp áo phần thân trên là khó nhất trong toàn bộ bài toán, và việc tăng capacity hay thêm convolution giúp thu hẹp vùng nhầm lẫn này chứ không loại bỏ hoàn toàn được nó. Ngược lại, các lớp có hình dạng đặc trưng rõ (Trouser, Bag, Sandal, Sneaker, Ankle boot) đều được cả ba mô hình nhận đúng trên 90%, kể cả với softmax tuyến tính đơn giản nhất.



Hình 7: Một số ảnh bị softmax dự đoán sai (T: nhãn thật, P: nhãn dự đoán)



Hình 8: Một số ảnh bị MLP dự đoán sai



Hình 9: Một số ảnh bị CNN dự đoán sai

Quan sát trực tiếp các ảnh dự đoán sai ở Hình 9 cho thấy phần lớn lỗi của CNN xảy ra ở những ảnh mà ngay cả người nhìn cũng khó phân biệt dứt khoát, ví dụ một chiếc áo khoác mỏng bị gán nhãn Coat nhưng mô hình dự đoán Pullover, hoặc một chiếc váy (Dress) ống rộng bị dự đoán thành Coat. Đây là giới hạn vốn có của dữ liệu (ambiguity giữa các lớp) hơn là lỗi của mô hình.

2.6 Nhận xét

Kết quả thực nghiệm cho thấy rõ sự đánh đổi giữa độ phức tạp kiến trúc và khả năng phân loại: softmax tuyến tính đơn giản, huấn luyện ổn định nhưng capacity quá thấp nên underfit, không tách được các lớp có ranh giới phi tuyến phức tạp. MLP tăng capacity giúp accuracy tăng đáng kể, nhưng vẫn coi ảnh như một vector phẳng, mất hoàn toàn thông tin cấu trúc không gian hai chiều của ảnh. CNN đạt accuracy cao nhất với số tham số thấp hơn MLP ở các tầng trích đặc trưng (nhờ weight sharing của convolution) và hội tụ nhanh hơn, vì kiến trúc của nó phù hợp tự nhiên với dữ liệu ảnh. Cả ba mô hình đều gặp khó khăn giống nhau ở cùng một cụm lớp trang phục phần thân trên, cho thấy giới hạn này đến từ chính dữ liệu chứ không phải từ một kiến trúc cụ thể nào.

2.7 Hạn chế

Hai điểm sau cần lưu ý khi đọc kết quả của E1. Thứ nhất, mỗi mô hình chỉ chạy một lần với một seed (42), nên chênh lệch nhỏ như 4,1 điểm phần trăm giữa CNN và MLP là thật, nhưng chênh lệch còn nhỏ hơn giữa các lớp dễ (ví dụ 97,7% so với 98,7% giữa MLP và CNN ở lớp dễ nhất) nằm trong khoảng có thể dao động nếu đổi seed; cần chạy nhiều seed và báo cáo mean cùng độ lệch chuẩn để kết luận chắc chắn hơn. Thứ hai, số epoch khác nhau giữa CNN (15) và softmax/MLP (20) được chọn theo lúc mỗi mô hình hội tụ trên validation, không phải một ngân sách epoch cố định cho cả ba; vì vậy nhận xét "CNN hội tụ nhanh hơn" chỉ đúng theo nghĩa cần ít epoch hơn để đạt validation accuracy tốt nhất của chính nó, chứ chưa phải một phép so sánh tốc độ hội tụ được kiểm soát chặt chẽ giữa ba kiến trúc.

3 E2: Transformer, MSA và tokenization

3.1 Mục tiêu và phạm vi

Bài E2 phân loại Fashion-MNIST bằng một bộ mã hóa Transformer nhỏ, không dùng mô hình pretrained. Báo cáo trả lời hai câu hỏi: (i) MSA tự viết và MSA của PyTorch khác nhau thế nào khi cùng dùng patch 4×4; (ii) cách biến ảnh thành chuỗi token ảnh hưởng ra sao khi cùng dùng MSA tự viết. Đề yêu cầu ít nhất hai cách lấy token; nhóm cài đặt bốn cách, tổng cộng năm thí nghiệm.

Trong MSA, mỗi token tổng hợp thông tin từ các token khác theo trọng số tính từ chính đầu vào. Ở bài này, token là vector mô tả một ô ảnh, một hàng ảnh hoặc một vị trí trên feature map. Cách dùng chuỗi patch lấy ý tưởng từ ViT [3], với mô hình nhỏ hơn nhiều.

3.2 Kiến trúc và công thức

Ảnh đầu vào có dạng $(B, 1, 28, 28)$, với B là số ảnh trong batch. Tokenizer tạo $(B, N, 64)$. Mô hình thêm một vector CLS học được vào đầu chuỗi và cộng vector vị trí học được, nên độ dài đi vào attention là $S = N + 1$.

Mô hình có bốn khối encoder dạng pre-norm:

$$\begin{aligned} u &= z + \mathrm{MSA}(\mathrm{LayerNorm}(z)) \\ z_{\text{next}} &= u + \mathrm{FFN}(\mathrm{LayerNorm}(u)) \\ \mathrm{FFN}(x) &= \mathrm{Linear}_2(\mathrm{Dropout}(\mathrm{GELU}(\mathrm{Linear}_1(x)))) \end{aligned}$$

FFN đổi chiều $64 \to 128 \to 64$ trên từng token, còn MSA trao đổi thông tin giữa các token. Cuối encoder, token CLS đi qua LayerNorm và Linear $64 \to 10$ để tạo logits. `CrossEntropyLoss` nhận logits trực tiếp.

Trong mỗi head, với chiều head $d = 64/4 = 16$ [2]:

$$\begin{aligned} Q = XW_Q + b_Q, \quad K = XW_K + b_K, \quad V = XW_V + b_V \\ A = \mathrm{softmax}(QK^\top/\sqrt{d}) \\ \mathrm{head} = AV \\ \mathrm{MSA}(X) = \mathrm{concat}(\mathrm{head}_1, \ldots, \mathrm{head}_4)W_O + b_O \end{aligned}$$

`ScratchMSA` gộp ba phép chiếu thành một `Linear(64, 192)`, tách Q/K/V thành bốn head, rồi dùng `einsum` để tính điểm và tổng có trọng số; bản này không gọi `nn.MultiheadAttention` hay `nn.TransformerEncoder`. `TorchMSA` gọi `nn.MultiheadAttention` với `batch_first=True`, truyền cùng x cho query, key, value [4].

Bảng 8: Kích thước tensor với patch4 ($N = 49$, $S = 50$)

Đại lượng	Kích thước
Ảnh	$B \times 1 \times 28 \times 28$
Patch đã trải phẳng	$B \times 49 \times 16$
Token sau Linear	$B \times 49 \times 64$
Chuỗi sau thêm CLS	$B \times 50 \times 64$
Mỗi Q, K, V sau tách head	$B \times 4 \times 50 \times 16$
Điểm / trọng số attention	$B \times 4 \times 50 \times 50$
Kết quả ghép head	$B \times 50 \times 64$
Logits phân loại	$B \times 10$

3.3 Bốn cách lấy token

Bảng 9: Bốn cách biến ảnh thành chuỗi token

Cách	Phần ảnh trước phép chiếu	N	S^2 mỗi head
`patch4`	Ô 4×4, stride 4	49	2.500
`patch7s3`	Ô 7×7, stride 3 (chồng lấn)	64	4.225
`row`	Một hàng gồm 28 pixel	28	841
`cnnstem`	Một vị trí trong feature map 7×7	49	2.500

Với patch, số ô theo một chiều là $\lfloor (28 - p)/s \rfloor + 1$, không padding. `patch4` cho lưới 7×7. `patch7s3` cho lưới 8×8 với các ô chồng lấn. `row` chiếu mỗi hàng $28 \to 64$, ít cặp attention nhất nhưng nén toàn bộ thông tin ngang của hàng vào một vector. `cnnstem` gồm Conv $1 \to 32$, BatchNorm, ReLU, MaxPool rồi Conv $32 \to 64$, BatchNorm, ReLU, MaxPool; feature map $64\times7\times7$ được trải thành 49 token. Vì có thêm các lớp convolution, thí nghiệm này thay đổi cả số tham số và loại phép biến đổi, ngoài cách chia token. Nhóm không dùng token theo từng pixel ($N = 784$) vì chi phí attention tăng theo N^2: khoảng 615 nghìn cặp cho mỗi ảnh và mỗi head.

3.4 Thiết kế thí nghiệm và huấn luyện

Bảng 10: Cấu hình dùng chung cho năm thí nghiệm

Thuộc tính	Giá trị
Chiều token / số khối / số head	64 / 4 / 4
FFN / dropout	$64 \to 128 \to 64$ / 0,1
Optimizer	AdamW
Learning rate cơ sở / weight decay	0,001 / 0,05
Epoch / batch / seed	30 / 128 / 42
Lịch learning rate	Warmup 2 epoch rồi cosine theo từng batch
Dữ liệu	Train 55.000, validation 5.000, test 10.000
Chọn checkpoint	Validation accuracy cao nhất

Training loop tự viết theo thứ tự forward → loss → `zero_grad` → backward → `optimizer.step` → `scheduler.step`. Khi validation, mô hình chuyển sang `eval` và tắt gradient. Mỗi thí nghiệm là một cặp (tokenizer, bản MSA); hai phép so sánh theo đề là (i) `patch4_scratch` với `patch4_torch`, và (ii) `patch4_scratch`, `patch7s3_scratch`, `row_scratch`, `cnnstem_scratch` với nhau.

3.5 Kiểm chứng MSA tự viết

Nhóm chép trọng số Q/K/V và đầu ra từ `TorchMSA` sang `ScratchMSA`, chuyển cả hai sang `eval`, cho cùng đầu vào rồi so output, attention từng head và gradient theo đầu vào. Ngưỡng đạt là 10^{-5} cho output và attention, 10^{-4} cho gradient; tổng mỗi hàng attention cũng phải bằng 1.

Bảng 11: Kết quả kiểm chứng MSA (`msa_check.json`)

Cấu hình (B, S, D, số head)	Max sai lệch output	Max sai lệch attention	Gradient đầu vào
8, 50, 64, 4	0,0	0,0	0,0
8, 29, 32, 8	0,0	0,0	0,0

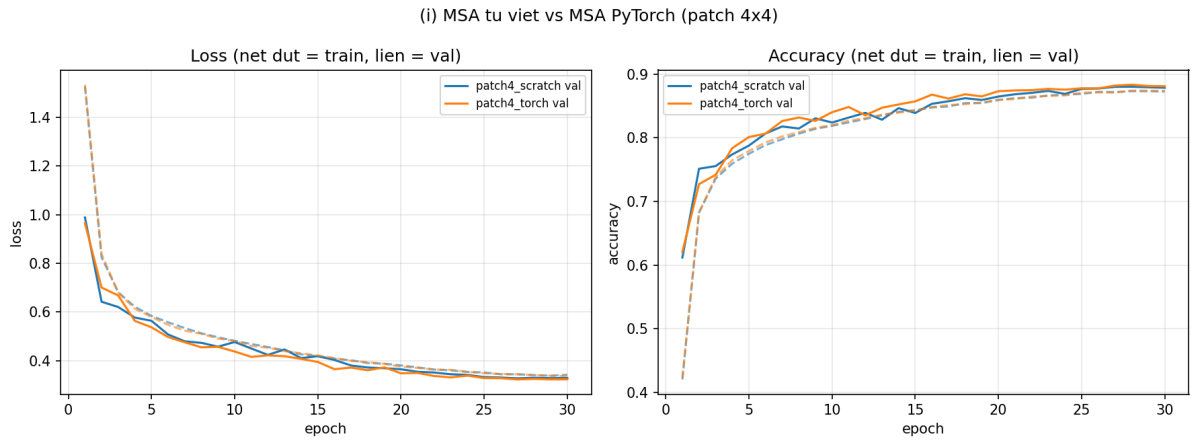
Cả hai cấu hình đều đạt. Phép kiểm tra chạy trên CPU, chế độ eval, không dùng dropout hay mask.

3.6 Kết quả so sánh

Bảng 12: Kết quả năm thí nghiệm trên tập test

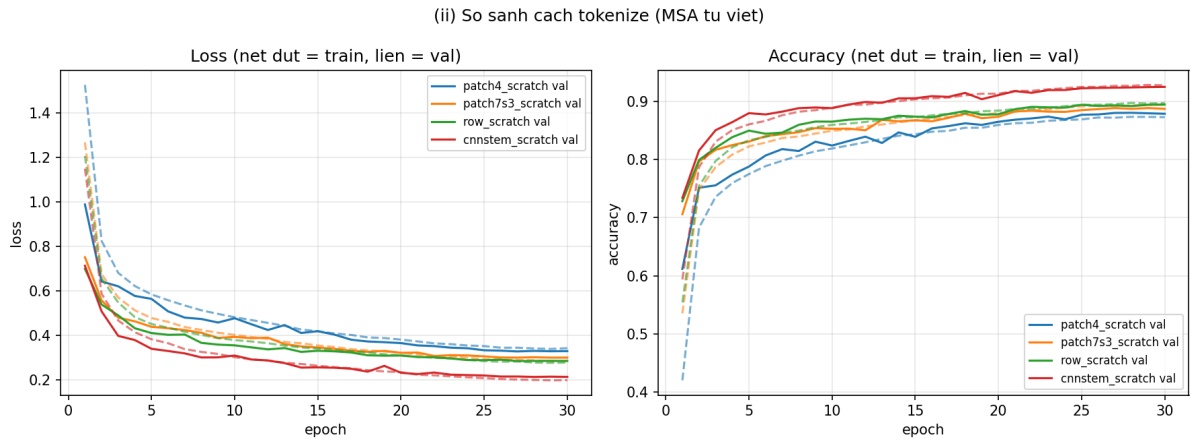
Thí nghiệm	Tham số	Val tốt nhất (%)	Test (%)	Macro-F1 (%)
patch4_scratch	139.018	88,00	88,50	88,42
patch4_torch	139.018	88,32	88,54	88,40
patch7s3_scratch	142.090	88,82	89,54	89,48
row_scratch	138.442	89,42	89,14	89,05
cnnstem_scratch	156.938	92,46	92,25	92,22

(i) MSA tự viết và PyTorch với cùng patch4. Hai mô hình có cùng số tham số và kết quả gần như nhau: test 88,50% và 88,54%, chênh 4 ảnh trên 10.000. Hai bản cũng khởi tạo khác nhau (`nn.MultiheadAttention` dùng Xavier, bản tự viết dùng mặc định của `nn.Linear`), nên với một seed, chênh lệch này chưa đủ để kết luận bản nào tốt hơn.



Hình 10: Đường cong loss và accuracy của hai bản MSA với patch4

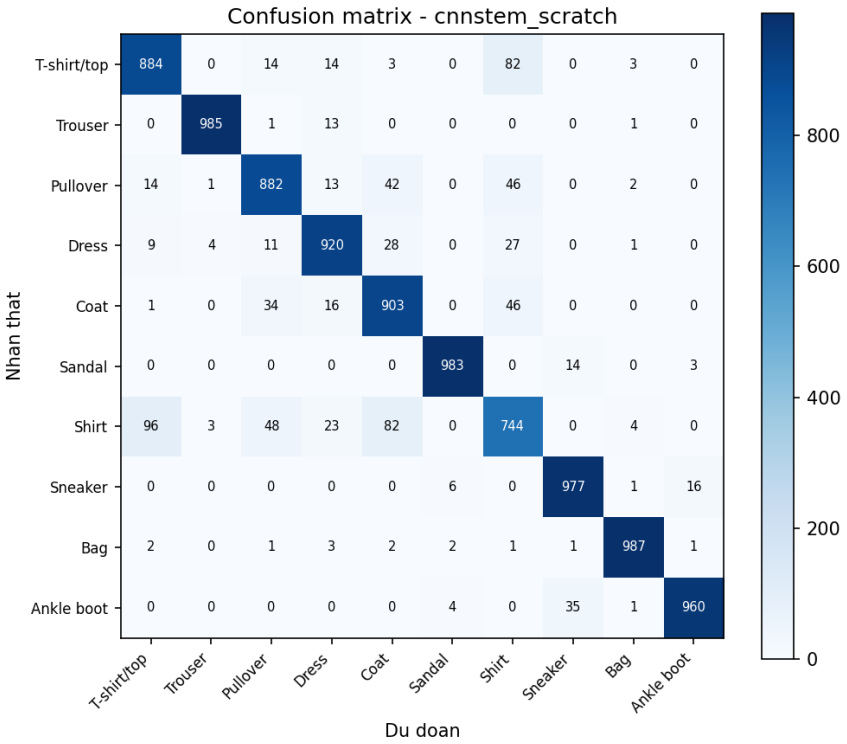
(ii) So sánh bốn cách lấy token (cùng MSA tự viết). `cnnstem` đạt test cao nhất, 92,25%, hơn `patch4` 3,75 điểm với thêm 17.920 tham số (12,89%); các lớp convolution giúp mô hình học mẫu cục bộ trước attention. `patch7s3` (patch chồng lấn) đạt 89,54%, hơn `patch4` 1,04 điểm. `row` đạt 89,14% với chỉ 28 token ảnh, cũng cao hơn `patch4` dù có ít token hơn — nhiều token hơn không đi kèm accuracy cao hơn trong thí nghiệm này.



Hình 11: Đường cong loss và accuracy của bốn tokenizer với MSA tự viết

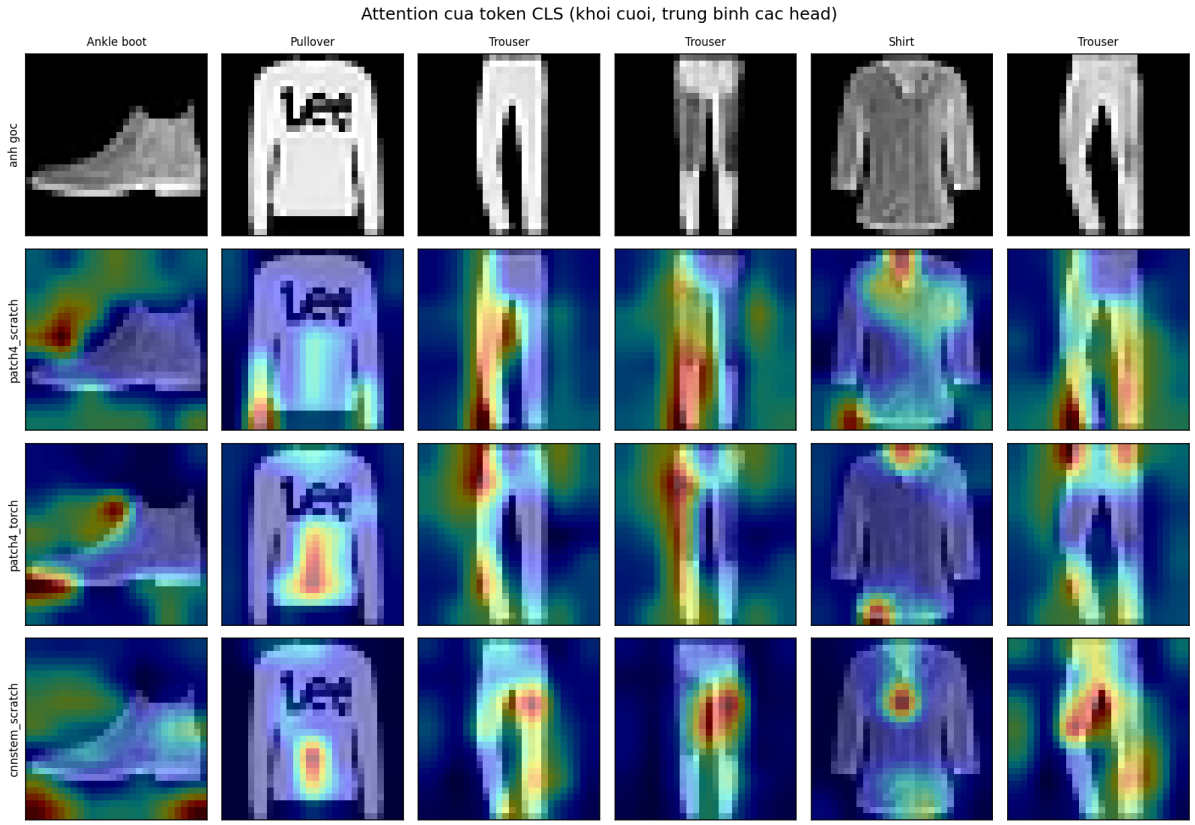
Ở epoch cuối, train accuracy và validation accuracy của cả năm mô hình chênh dưới 1 điểm, ví dụ `cnnstem` đạt 92,72% và 92,46%. Epoch tốt nhất đều từ 27 trở đi; các mô hình chưa có dấu hiệu overfit rõ, nên có thể thử mô hình lớn hơn hoặc train lâu hơn.

3.7 Phân tích lỗi và hình attention



Hình 12: Confusion matrix của `cnnstem_scratch`, mô hình có validation accuracy cao nhất

Hàng là nhãn thật, cột là dự đoán. Mô hình đúng 9.225 và sai 775 ảnh trên 10.000. Các ô nhầm lớn nhất là Shirt → T-shirt/top (96 ảnh), T-shirt/top → Shirt (82 ảnh) và Shirt → Coat (82 ảnh). Shirt có recall thấp nhất, 74,40%, trong khi Bag đạt 98,70%. Ngoài Shirt, lỗi còn tập trung giữa Pullover và Coat. Kết quả này khớp với nhận xét ở mục 1.2 rằng các lớp áo có đường viền giống nhau.



Hình 13: Trọng số từ token CLS đến token ảnh ở khối cuối, trung bình các head

Hình này lấy hàng query CLS, bỏ cột CLS, trung bình bốn head rồi nội suy lên 28×28. Vùng nóng khác nhau khá nhiều giữa `patch4_scratch` và `patch4_torch` trên cùng một ảnh, dù hai mô hình có accuracy gần bằng nhau. Với `patch4`, một số vùng nóng nằm ngoài vật thể. Với `cnnstem`, vùng nóng tập trung ở phần giữa vật thể hơn. Hình chỉ cho biết CLS đặt trọng số ở đâu, chưa cho biết vùng đó quyết định dự đoán.

3.8 Kết luận E2

Nhóm đã cài đặt MSA tự viết, MSA của PyTorch và bốn cách lấy token. MSA tự viết khớp với bản PyTorch trong cả hai phép kiểm tra, và accuracy của hai bản gần như nhau. `cnnstem` cho kết quả cao nhất cả trên validation lẫn test; `row` và patch chồng lấn nhỉnh hơn `patch4`. Lỗi chủ yếu nằm ở các lớp áo, nhất là Shirt.

Kết quả dựa trên một seed và một lần chạy cho mỗi cấu hình, các tokenizer chưa được cân bằng số tham số, và thời gian chưa được đo riêng cho MSA. Bước tiếp theo là lặp lại với nhiều seed và đo riêng thời gian nạp dữ liệu.

4 So sánh E1 và E2

CNN của E1 và `cnnstem` của E2 có cùng hai khối convolution (19.008 tham số). Điểm khác là phần phía sau: E1 trải feature map rồi đưa qua các lớp Linear, còn E2 coi mỗi vị trí trên feature map là một token và đưa qua bốn khối MSA. Vì vậy, `cnnstem` có thể xem như một cách thay phần phân loại của CNN bằng MSA.

Bảng 13: CNN (E1) so với cnnstem (E2)

Chỉ số	CNN (E1)	CNN-stem (E2)
Tổng tham số	421.834	156.938
Tham số sau phần convolution	402.826	137.930
Test accuracy (%)	91,45	92,25
Macro-F1 (%)	91,39	92,22
Số ảnh test sai	855	775
Recall của Shirt (%)	69,60	74,40
Shirt → T-shirt/top (ảnh)	117	96
Shirt → Coat (ảnh)	97	82

`cnnstem` đúng thêm 80 ảnh test với số tham số bằng khoảng 37% của CNN. Phần lớn cải thiện về Shirt đến từ việc giảm nhầm sang T-shirt/top và Coat, trong khi T-shirt/top → Shirt gần như không đổi (81 và 82 ảnh). Attention cho phép mô hình tổng hợp thông tin giữa các vị trí xa nhau trên ảnh, nên nhóm cho rằng đây có thể là một lý do của cải thiện này.

Kết quả này chưa đủ để quy toàn bộ cải thiện cho MSA, vì E2 còn dùng AdamW, warmup và cosine, train 30 epoch so với 15 epoch của CNN, và mỗi mô hình mới chạy một seed. Để kiểm tra riêng tác động của MSA, cần train hai mô hình với cùng optimizer, lịch learning rate và số epoch.

Nếu bỏ convolution, kết quả thấp hơn: ba cách lấy token còn lại đạt 88,50% đến 89,54%, cao hơn MLP (87,39%) nhưng thấp hơn CNN (91,45%). Với tập dữ liệu và kích thước mô hình này, MSA cho kết quả tốt nhất khi đi sau các lớp convolution, chứ chưa thay thế được convolution.

5 E3: LSTM và GRU trên chuỗi ảnh

5.1 Mục tiêu

Bài E3 mô hình hóa ảnh Fashion-MNIST như dữ liệu chuỗi, dùng LSTM và GRU để phân loại, rồi so với baseline không hồi quy (MLP, CNN) đã có từ E1 trên cùng split. Mục tiêu là xem recurrent network, vốn thiết kế cho dữ liệu tuần tự một chiều, xử lý ảnh hai chiều tốt đến đâu so với mạng coi ảnh là vector phẳng (MLP) hay mạng có inductive bias không gian (CNN).

5.2 Hai cách biến ảnh thành chuỗi

Bảng 14: Hai cách biến ảnh 28×28 thành chuỗi token

Cách	Mô tả	Số bước (T)	Chiều mỗi bước
`row`	Mỗi hàng pixel là một bước thời gian	28	28
`patch`	Ô 4×4 không chồng lấn, quét theo hàng	49	16

`row` giữ nguyên toàn bộ một hàng ảnh làm vector đầu vào ở mỗi bước, nên mỗi bước có ngữ cảnh ngang đầy đủ của hàng đó. `patch` chia ảnh thành lưới 7×7 ô 4×4, trải phẳng mỗi ô thành vector 16 chiều rồi quét từ trái sang phải, trên xuống dưới; mỗi bước chỉ thấy một vùng cục bộ nhỏ, mô hình phải tự tích lũy ngữ cảnh không gian qua 49 bước hồi quy. Cả hai cách đều không chồng lấn và không học thêm phép chiếu token (khác với `patch4` của E2, vốn có thêm một lớp Linear chiếu token); dữ liệu đưa thẳng vào LSTM/GRU.

5.3 Kiến trúc

Cả LSTM và GRU dùng `nn.LSTM`/`nn.GRU` của PyTorch, một tầng, hidden size 128, lấy hidden state ở bước cuối cùng làm đặc trưng cho head phân loại `Linear(128, 10)`. Số tham số:

$$\text{LSTM} = 4 \times \text{hidden} \times (\text{input} + \text{hidden} + 2) + (\text{hidden} \times 10 + 10)$$
$$\text{GRU} = 3 \times \text{hidden} \times (\text{input} + \text{hidden} + 2) + (\text{hidden} \times 10 + 10)$$

LSTM có 4 cổng (input, forget, cell, output), GRU có 3 cổng (reset, update, candidate), nên với cùng hidden size và cùng đầu vào, GRU luôn ít tham số hơn LSTM khoảng 25%.

Bảng 15: Số tham số của bốn cấu hình

Mô hình	Cách biến chuỗi	Số tham số
LSTM	`row`	82.186
GRU	`row`	61.962
LSTM	`patch`	76.042
GRU	`patch`	57.354

Cả bốn cấu hình có số tham số nhỏ hơn nhiều so với MLP (235.146) và CNN (421.834) của E1, vì trọng số LSTM/GRU được chia sẻ qua mọi bước thời gian (giống cơ chế weight sharing của convolution qua mọi vị trí không gian).

5.4 Cấu hình huấn luyện

Bảng 16: Cấu hình huấn luyện dùng chung cho bốn cấu hình

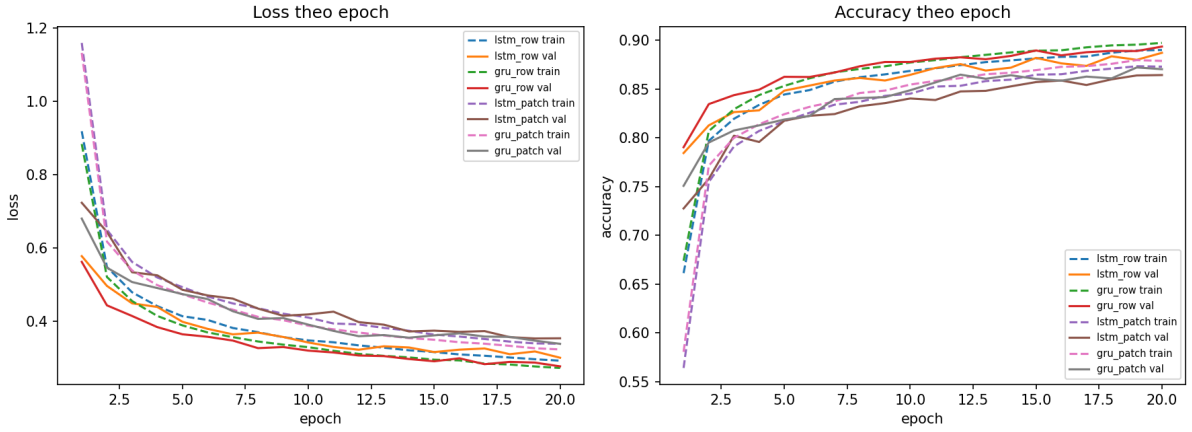
Thuộc tính	Giá trị
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epoch	20
Seed	42
Dữ liệu	Train 55.000, validation 5.000, test 10.000 (giống E1)
Chọn checkpoint	Validation accuracy cao nhất

Training loop tự viết theo thứ tự forward → loss → `optimizer.zero_grad()` → `loss.backward()` → `optimizer.step()`, giống vòng lặp đã dùng ở E1. Baseline MLP và CNN không huấn luyện lại; nhóm load checkpoint `best.pt` đã có từ E1 và chỉ chạy lại đánh giá trên tập test, đảm bảo dùng đúng cùng split và cùng điều kiện so sánh.

5.5 Kết quả

Bảng 17: So sánh bốn cấu hình LSTM/GRU với baseline MLP/CNN trên tập test

Mô hình	Chuỗi	Số tham số	Val acc. (%)	Test acc. (%)	Thời gian (giây)
LSTM	row	82.186	88,72	88,55	309,1
GRU	row	61.962	89,36	88,95	296,3
LSTM	patch	76.042	86,44	86,89	294,7
GRU	patch	57.354	87,22	86,75	291,6
MLP (E1)	–	235.146	87,32	87,39	299,7
CNN (E1)	–	421.834	91,76	91,45	236,2



Hình 14: Đường cong loss và accuracy theo epoch của bốn cấu hình LSTM/GRU (nét đứt: train, nét liền: validation)

row so với patch. Với cùng kiến trúc, `row` luôn cho test accuracy cao hơn `patch`: LSTM 88,55% so với 86,89% (+1,66 điểm), GRU 88,95% so với 86,75% (+2,20 điểm). Mỗi bước của `row` có ngữ cảnh một hàng đầy đủ (28 chiều), còn mỗi bước của `patch` chỉ có một ô 4×4 cục bộ (16 chiều); mô hình phải tự ghép nối ngữ cảnh không gian qua nhiều bước hồi quy hơn (49 so với 28 bước), nên khó học hơn.

LSTM so với GRU. Kết quả không đồng nhất giữa hai cách biểu diễn. Với `row`, GRU tốt hơn LSTM (88,95% so với 88,55%, +0,40 điểm) dù ít hơn 25% tham số. Với `patch`, LSTM nhỉnh hơn GRU một chút (86,89% so với 86,75%, +0,14 điểm) cũng dù GRU ít tham số hơn. Chênh lệch ở cả hai trường hợp đều nhỏ và chỉ chạy một seed, nên chưa đủ để kết luận kiến trúc nào luôn tốt hơn; điểm rõ ràng hơn là GRU đạt accuracy tương đương hoặc nhỉnh hơn LSTM với ít tham số hơn đáng kể, nên hiệu quả hơn về capacity.

So với baseline MLP/CNN. Hai cấu hình `row` (88,55% và 88,95%) vượt MLP (87,39%) dù có ít tham số hơn MLP nhiều lần (82.186 và 61.962 so với 235.146), còn hai cấu hình `patch` (86,89% và 86,75%) lại thấp hơn MLP. Không cấu hình LSTM/GRU nào vượt được CNN (91,45%). Điều này cho thấy khả năng "bắt phụ thuộc thời gian" của LSTM/GRU giúp tích lũy ngữ cảnh không gian tốt hơn một MLP phẳng khi chuỗi đủ thông tin mỗi bước (`row`), nhưng cơ chế hồi quy tuần tự một chiều vẫn không thay thế được inductive bias không gian hai chiều của convolution: CNN xử lý đồng thời các vùng lân cận theo cả hai trục, trong khi LSTM/GRU phải truyền thông tin qua nhiều bước mới liên kết được các phần ảnh ở xa nhau theo chiều dọc.

5.6 Hạn chế

Kết quả E3 dựa trên một seed (42) và một lần chạy cho mỗi cấu hình, nên các chênh lệch nhỏ (dưới 0,5 điểm phần trăm, như giữa LSTM và GRU) chưa đủ để kết luận chắc chắn, tương tự lưu ý đã nêu ở mục 2.7 và 3.6. Nhóm cũng chưa làm phân tích lỗi theo từng lớp (confusion matrix) cho E3 như đã làm ở E1 và E2, do giới hạn thời gian; đây là phần có thể bổ sung sau. Baseline MLP/CNN lấy lại từ checkpoint E1 nên không cùng epoch/optimizer với LSTM/GRU (xem mục 2.3 và bảng trên), giống hạn chế đã nêu ở mục 4 khi so E1 với E2.

Phụ lục

Nguồn số liệu

Bảng 18: Vị trí số liệu dùng trong báo cáo, trong repository

Nội dung	File trong repo
Split dùng chung	`E1/splits/train_idx.npy, val_idx.npy, split_meta.json`
Môi trường và output lúc chạy	`E1/notebooks/E1_Colab.ipynb, E2/notebooks/E2_Colab.ipynb, E3/notebooks/E3_Colab.ipynb`
Kết quả E1	`E1/results/comparison_table.json; E1/results/<model>/history.csv, summary.json`
Cấu hình E2	`E2/src/experiments.py`
Đối chiếu MSA	`E2/results/msa_check.json`
Kết quả E2	`E2/results/comparison_table.json; E2/results/<tên thí nghiệm>/history.csv, summary.json`
Cách biến chuỗi của E3	`E3/src/sequences.py`
Kết quả E3	`E3/results/comparison_table.json; E3/results/<tên cấu hình>/history.csv, summary.json`
Baseline MLP/CNN của E3	`E3/results/baseline_table.json` (đánh giá lại checkpoint E1)

Khai báo hỗ trợ AI

Nhóm dùng công cụ AI (Claude Code) để hỗ trợ viết code khung cho E1, E2 và E3 (`data.py`, `models.py`, `train.py`, `evaluate.py` và các file tương ứng như `tokenizers.py`, `attention.py` của E2, `sequences.py`, `baseline.py` của E3) cùng ví dụ sử dụng, và gợi ý các thí nghiệm cần làm. Nhóm sửa đổi, bổ sung thêm vào template rồi tự chạy thí nghiệm trên Colab để lấy số liệu. Sau khi có kết quả, AI hỗ trợ tổng hợp và viết báo cáo, nhóm đọc và sửa lại sau đó. AI cũng hỗ trợ viết test, vẽ sơ đồ, dàn trang PDF và chỉnh README. Nhóm đã đọc, hiểu và kiểm tra lại toàn bộ code trước khi chạy, và chịu trách nhiệm về tính đúng đắn của số liệu cũng như nội dung phân tích.

Tài liệu tham khảo

[1] Han Xiao, Kashif Rasul, Roland Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms* (2017). https://github.com/zalandoresearch/fashion-mnist.

[2] Ashish Vaswani và cộng sự. *Attention Is All You Need*. NeurIPS 2017. https://arxiv.org/abs/1706.03762.

[3] Alexey Dosovitskiy và cộng sự. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR 2021. https://arxiv.org/abs/2010.11929.

[4] PyTorch. *MultiheadAttention*, tài liệu chính thức. https://pytorch.org/docs/stable/generated/torch.nn.MultiheadAttention.html.